

# Uninformed Search – cont'd

Instructor: Hyunwoo J. Kim

# Coding assignment #1

## CS470 – Coding Assignment #1

---

**Python Version: 3.9.**

### **Table of Contents**

- [1. Tutorial: Getting Familiar with the Environment!](#)
- [2. Coding Assignment #1 \(6 questions\)](#)
- [3. Submission policy](#)
- [4. Q&A \(TA information\)](#)

### **Evaluation**

**[100 pts] Code** will be graded by autograder (Q1, Q2, Q3, Q4) 25 points for each.

**[100 pts] Report** will be graded by TA (Q1, Q2, Q3, Q4, AQ1, AQ2)

# Verification or robust analysis

 $x$ 

“panda”

57.7% confidence

 $+ .007 \times$  $\text{sign}(\nabla_x J(\theta, x, y))$ 

“nematode”

8.2% confidence

 $=$  $x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$ 

“gibbon”

99.3 % confidence

**Additional Question 2 (40 report points): Make your own question that AI cannot solve!**

Attendance Code:

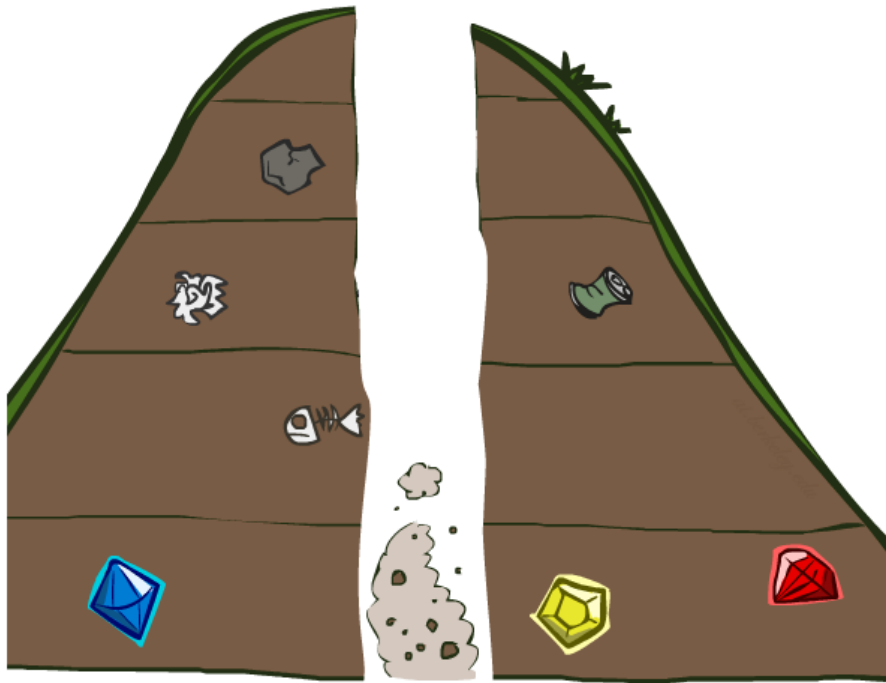
$$\max_{\delta} \mathcal{L}(f_{\theta}(x + \delta), y) \quad \text{s.t.} \quad \|\delta\|_p \leq \epsilon$$

This equation is used to understand AQ2 at a high level. To analyze the robustness of AI tools, we will challenge the AIs by modifying the problem  $x$  minimally ( $\|\delta\|_p < \epsilon$ ) so that the AI  $f_{\theta}(\cdot)$  cannot solve it correctly (increase  $L(\cdot, y)$ ).

If you have not used AI for the questions (Q1~Q4) above, pick one question above and complete it with your favorite AI.

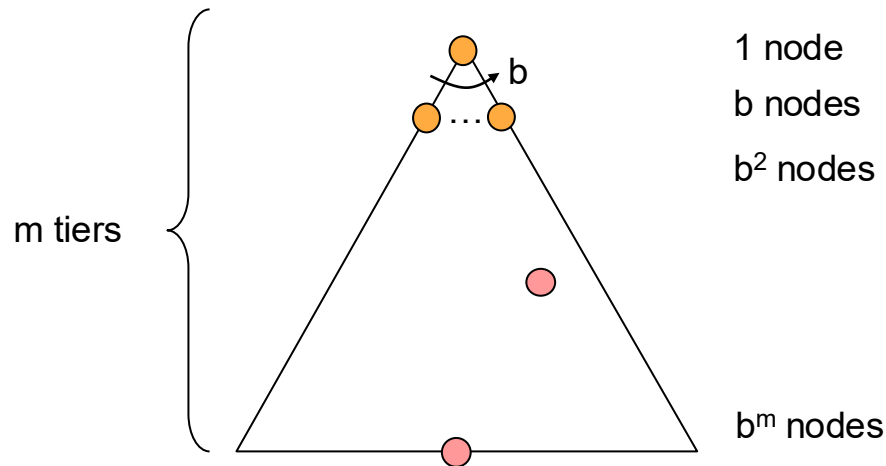
1. Vibe coding with your favorite AI (e.g., chatGPT, codex, Copilot, Cursor... )!
2. Analyze failure cases of your AI assistant.
3. If your AI successfully helps you with coding, then analyze how reliable the AI assistant by minimally perturbing ( $+\delta$  s.t.  $\|\delta\|_p < \epsilon$ ) the original question ( $x$ )

# Search Algorithm Properties



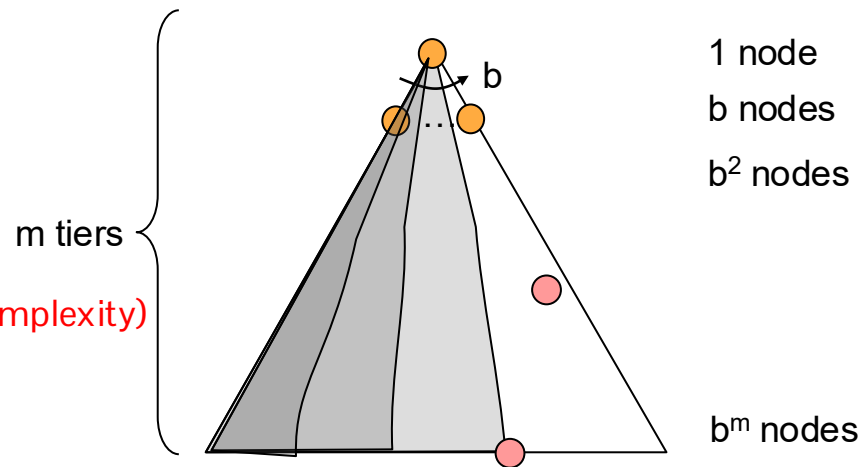
# Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?
- Cartoon of search tree:
  - $b$  is the branching factor
  - $m$  is the maximum depth
  - solutions at various depths
- Number of nodes in entire tree?
  - $1 + b + b^2 + \dots + b^m = O(b^m)$

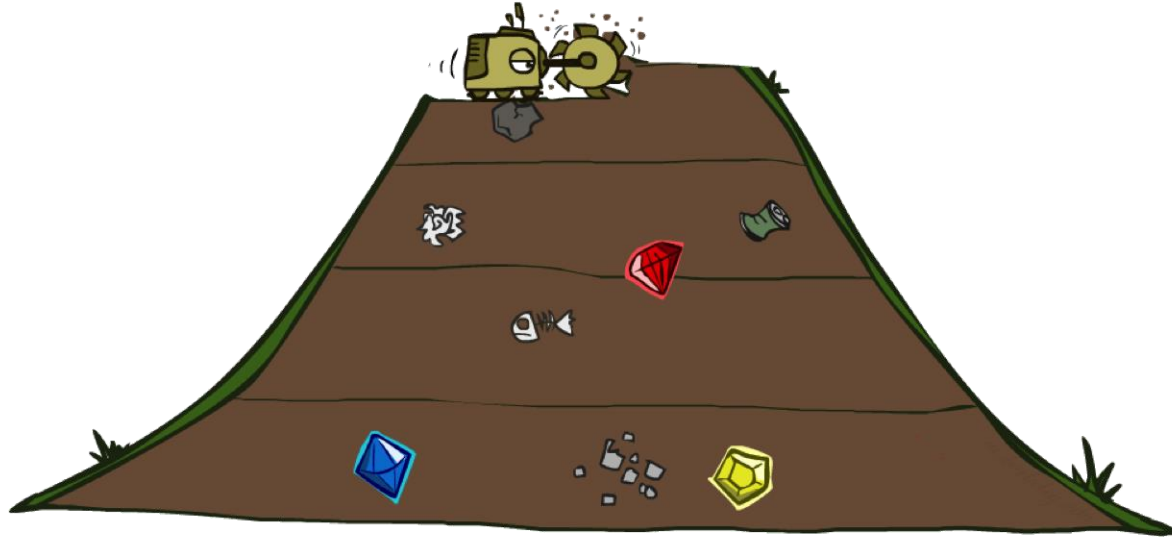


# Depth-First Search (DFS) Properties

- What nodes DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree (Time complexity)
  - If  $m$  is finite, takes time  $O(b^m)$
- How much space does the fringe take? (Space complexity)
  - Only has siblings on path to root, so  $O(bm)$
- Is it complete?
  - $m$  could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
  - No, it finds the “leftmost” solution, regardless of depth or cost



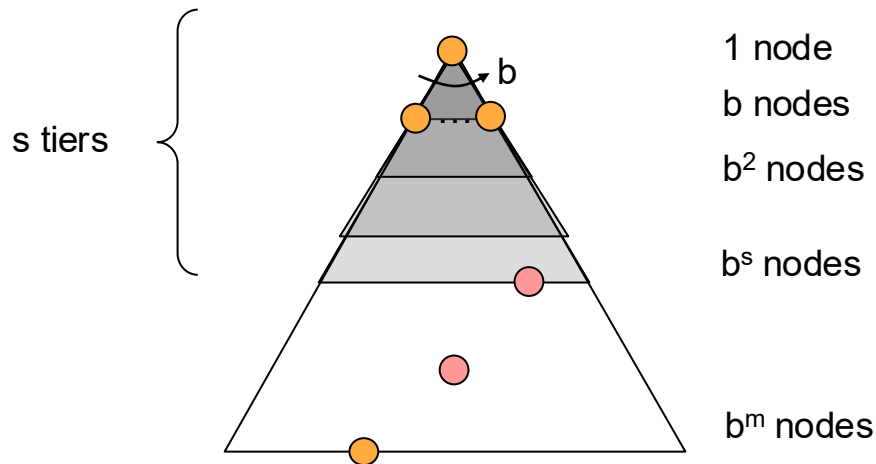
# Breadth-First Search



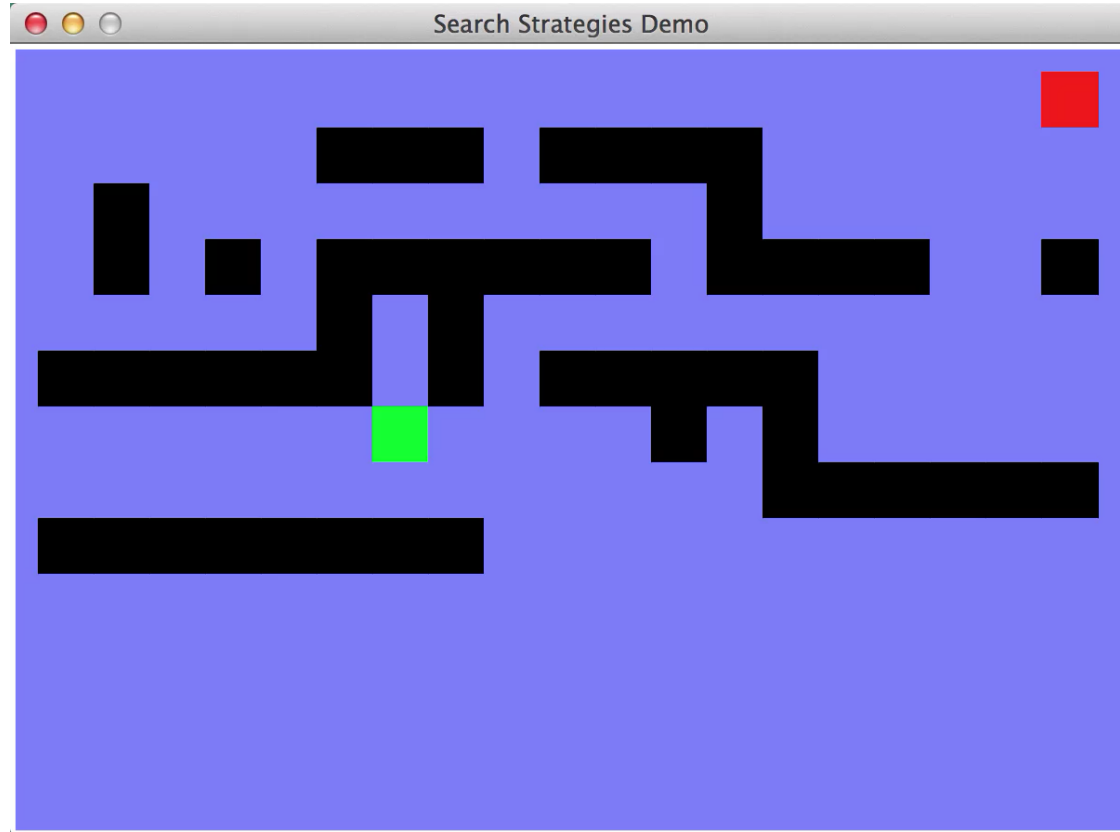


# Breadth-First Search (BFS) Properties

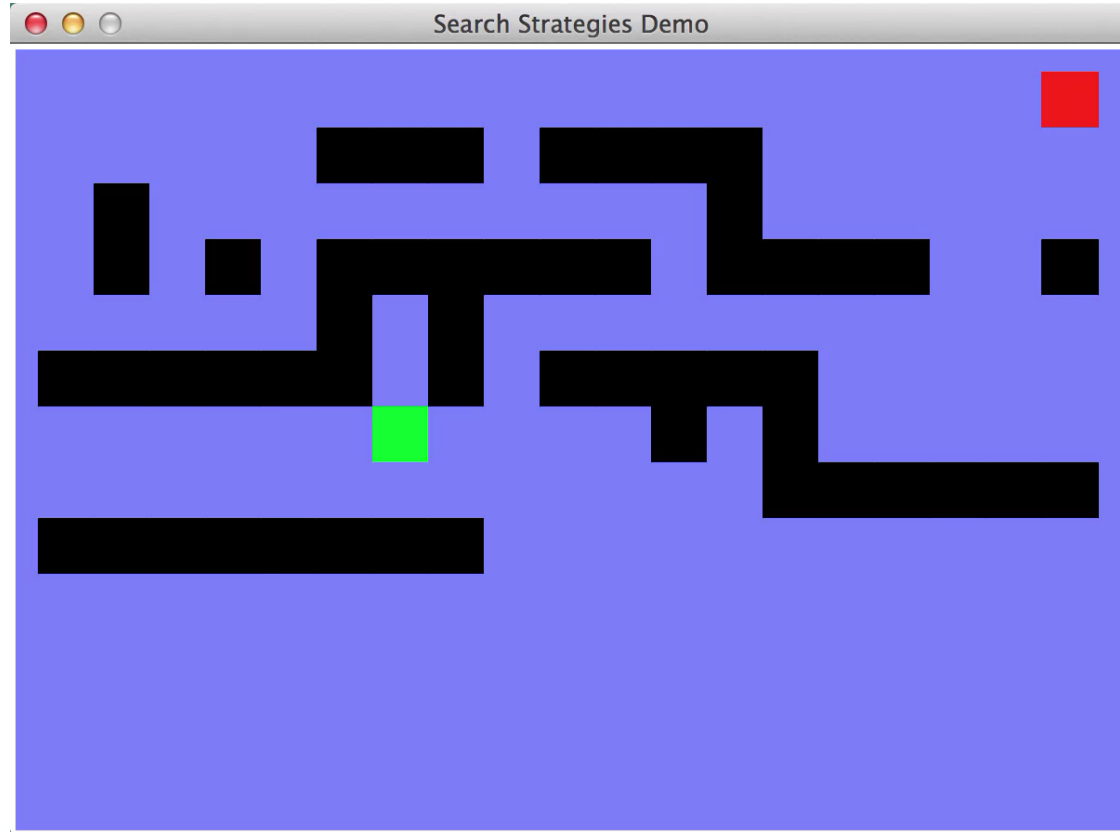
- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let **depth of shallowest solution** be  $s$
  - Search takes time  $O(b^s)$
- How much space does the fringe take?
  - Has roughly the last tier, so  $O(b^s)$
- Is it complete?
  - $s$  must be finite if a solution exists, so yes!
- Is it optimal?
  - Only if **costs are all 1** (more on costs later)



# Video of Demo Maze Water DFS/BFS

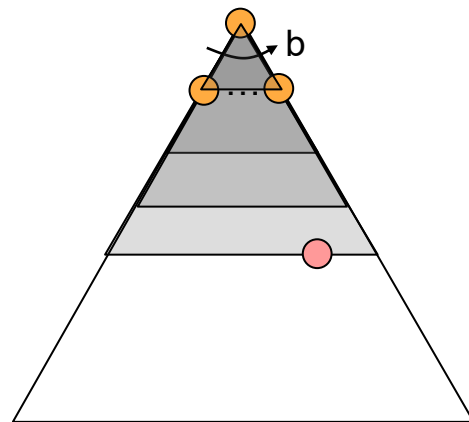


# Video of Demo Maze Water DFS/BFS



# Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution...
  - Run a DFS with depth limit 2. If no solution...
  - Run a DFS with depth limit 3. ....
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



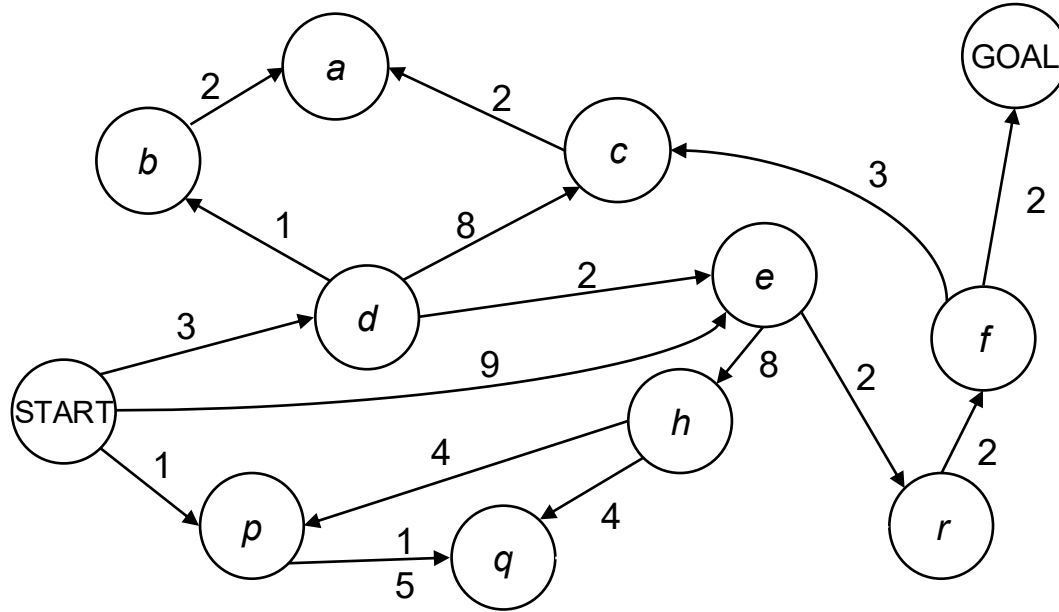
# Today's agenda

- Uniform Cost Search

Attendance Code:

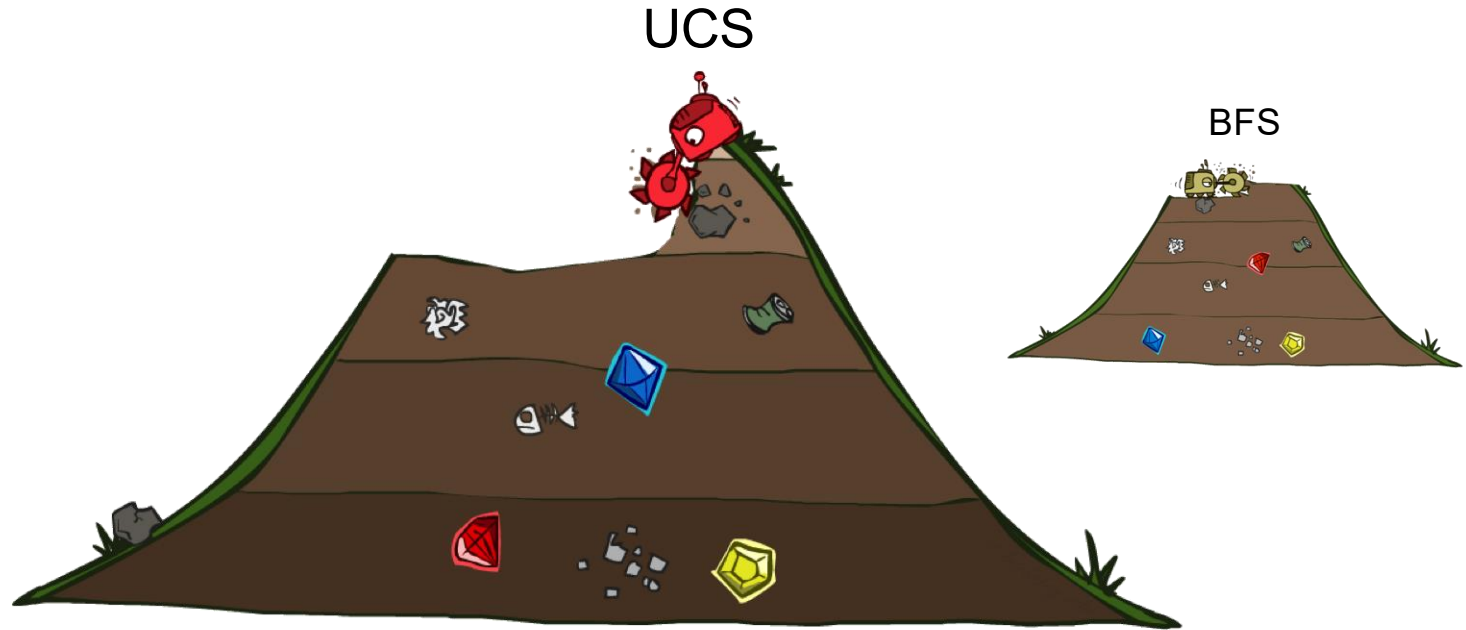
123

# Cost-Sensitive Search



BFS finds the shortest path in terms of **number of actions**. It does NOT find the **least-cost path**. We will now cover a similar algorithm which does find the least-cost path.

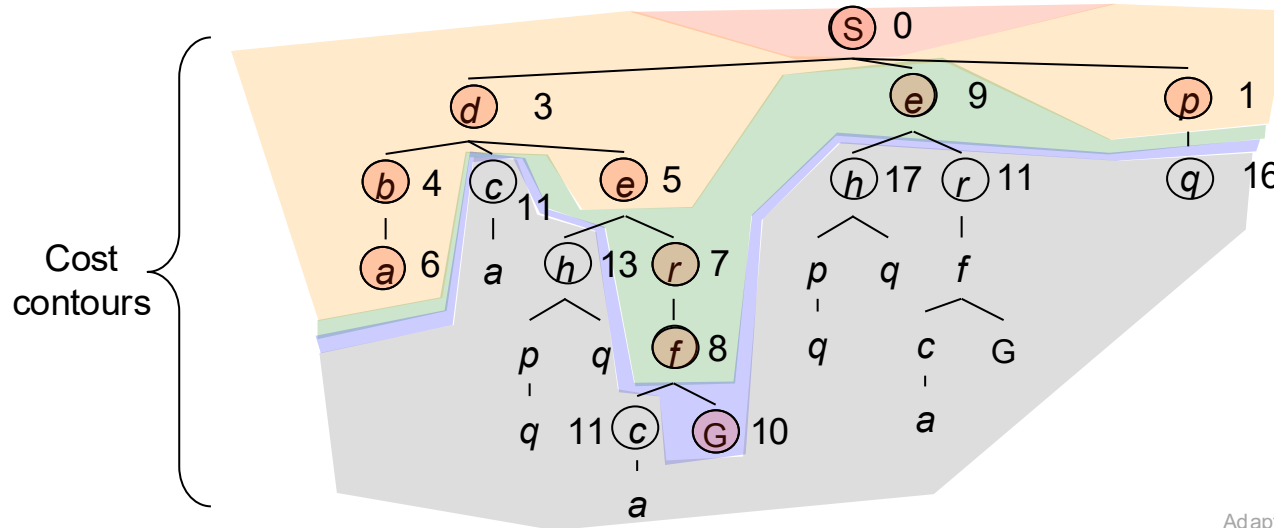
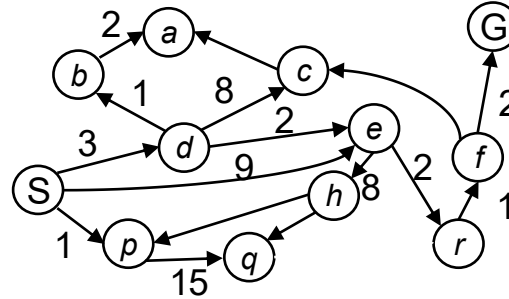
# Uniform Cost Search



# Uniform Cost Search

Strategy: expand a  
cheapest node first:

Fringe is a priority queue  
(priority: **cumulative cost**)

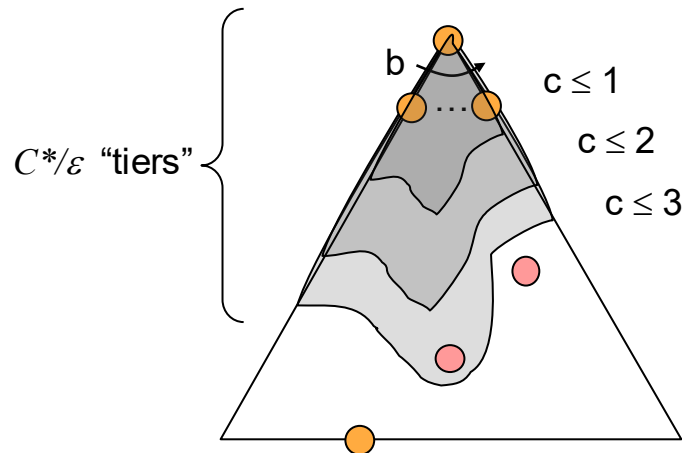




# Uniform Cost Search (UCS) Properties

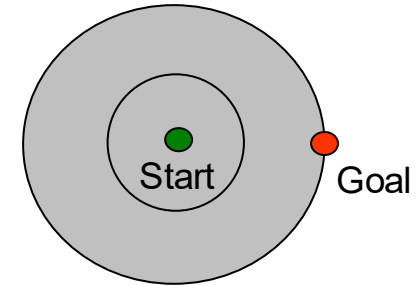
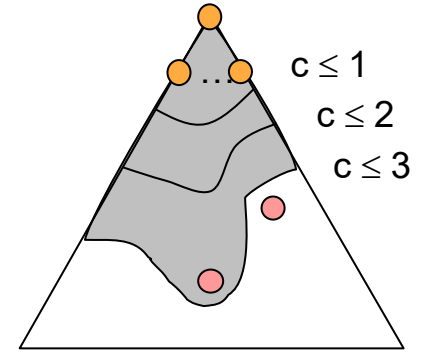
Attendance Code:  
123

- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs  $C^*$  and arcs cost at least  $\varepsilon$ , then the “effective depth” is roughly  $C^*/\varepsilon$
  - Takes time  $O(b^{C^*/\varepsilon})$  (exponential in effective depth)
- How much space does the fringe take?
  - Has roughly the last tier, so  $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - Yes! (Proof next lecture via  $A^*$ )



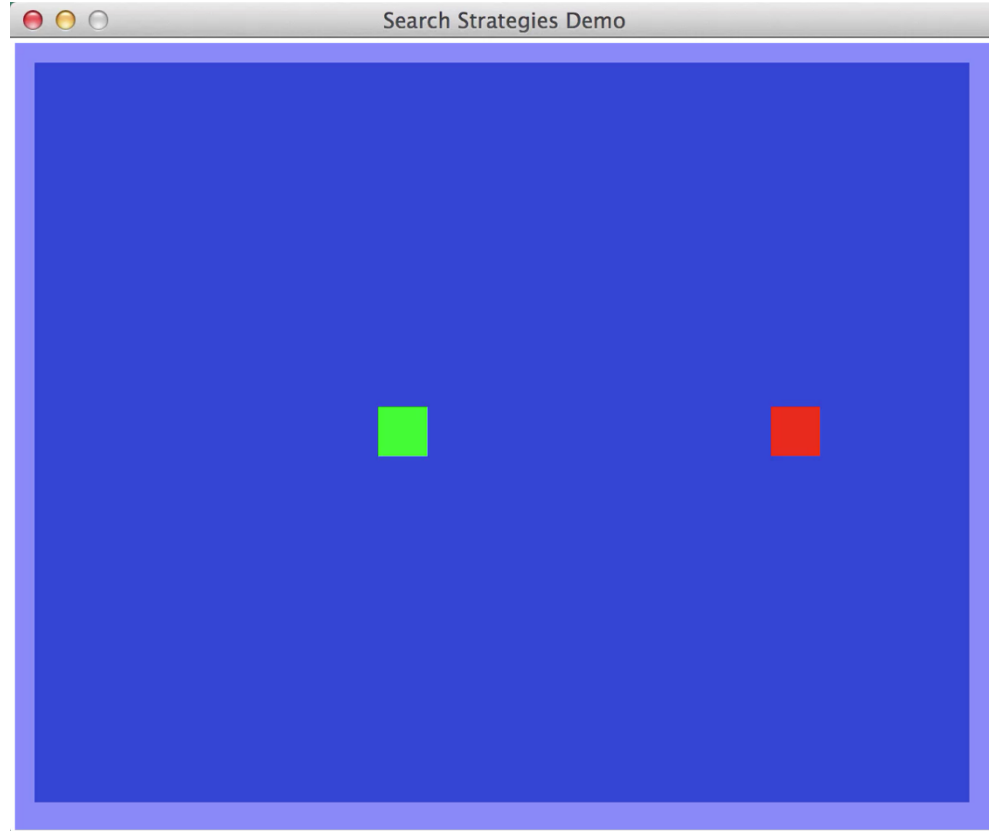
# Uniform Cost Issues

- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
  - Explores options in every “direction”
  - No information about goal location
- We'll fix that soon!



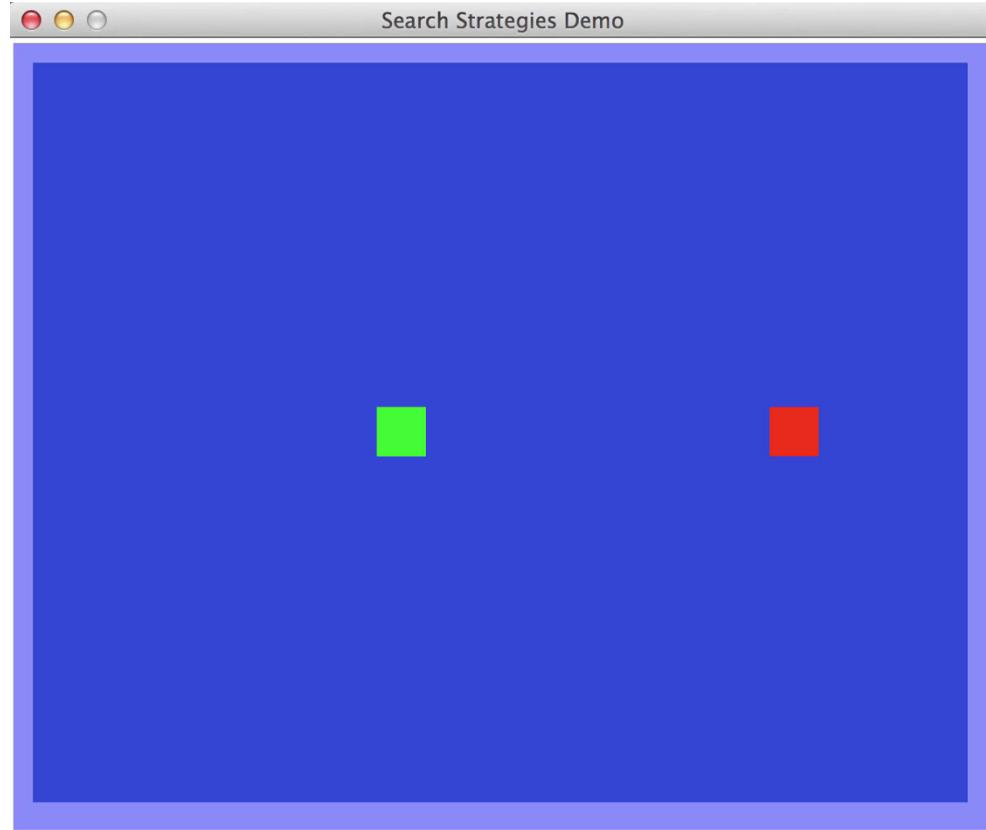
# Video of Demo Empty UCS

Attendance Code:  
123



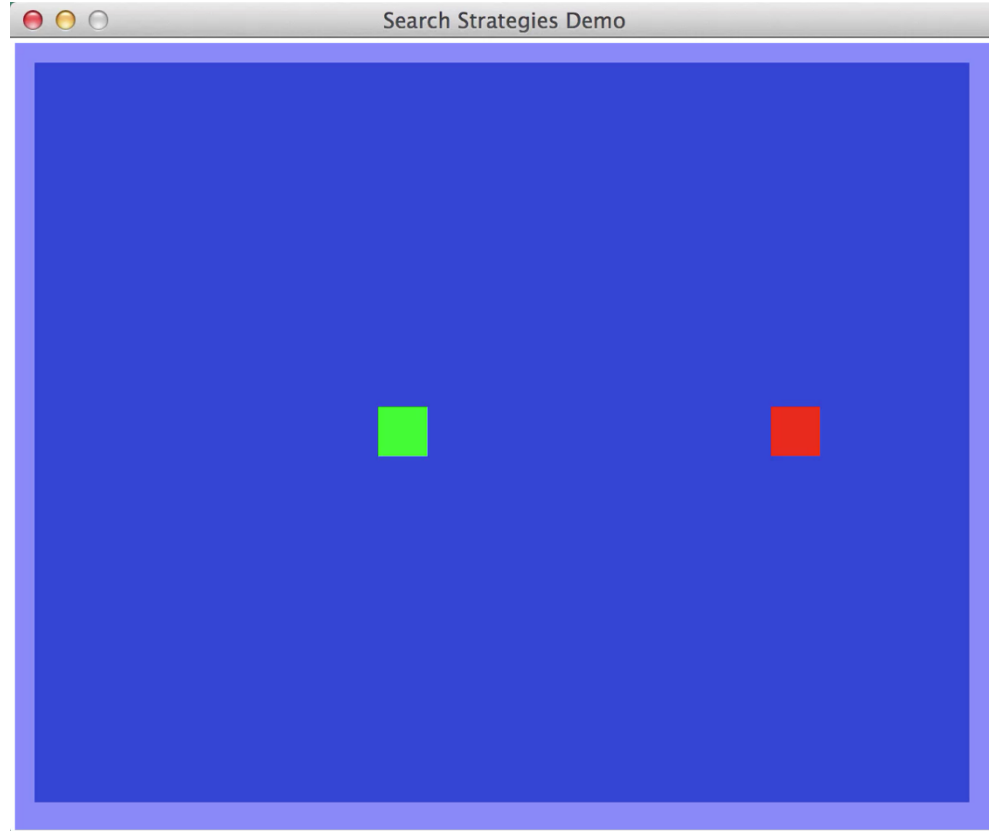
# Video of Demo Empty BFS

Attendance Code:  
123



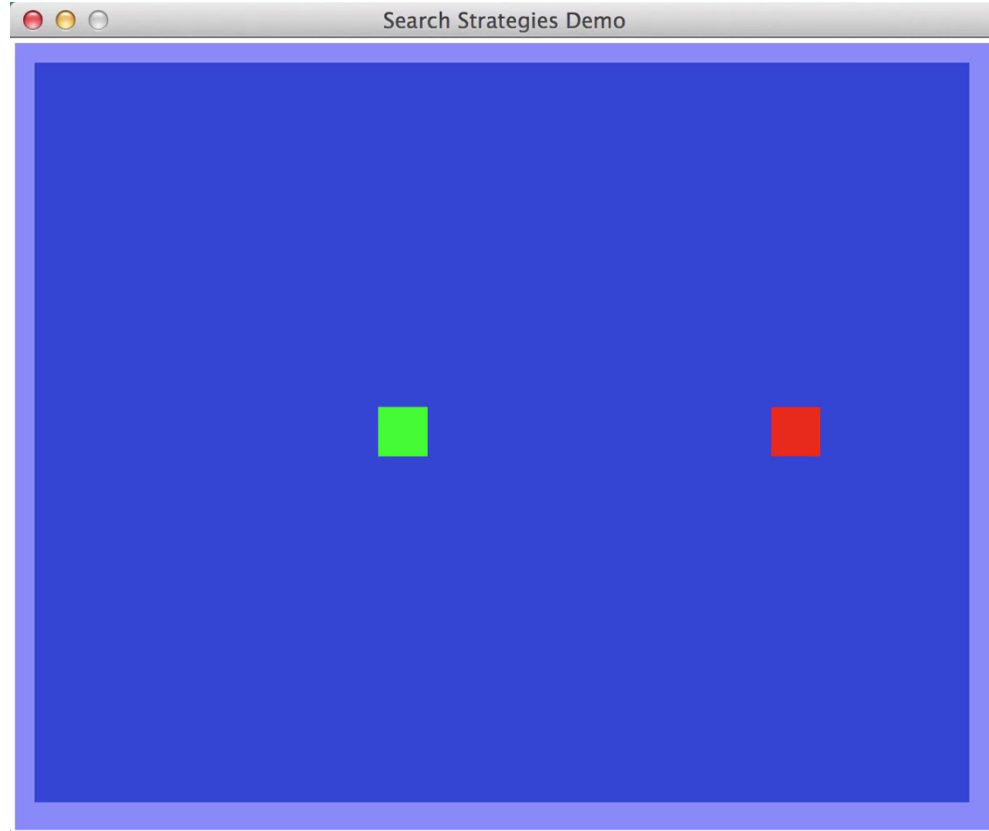
# Video of Demo Empty DFS

Attendance Code:  
123



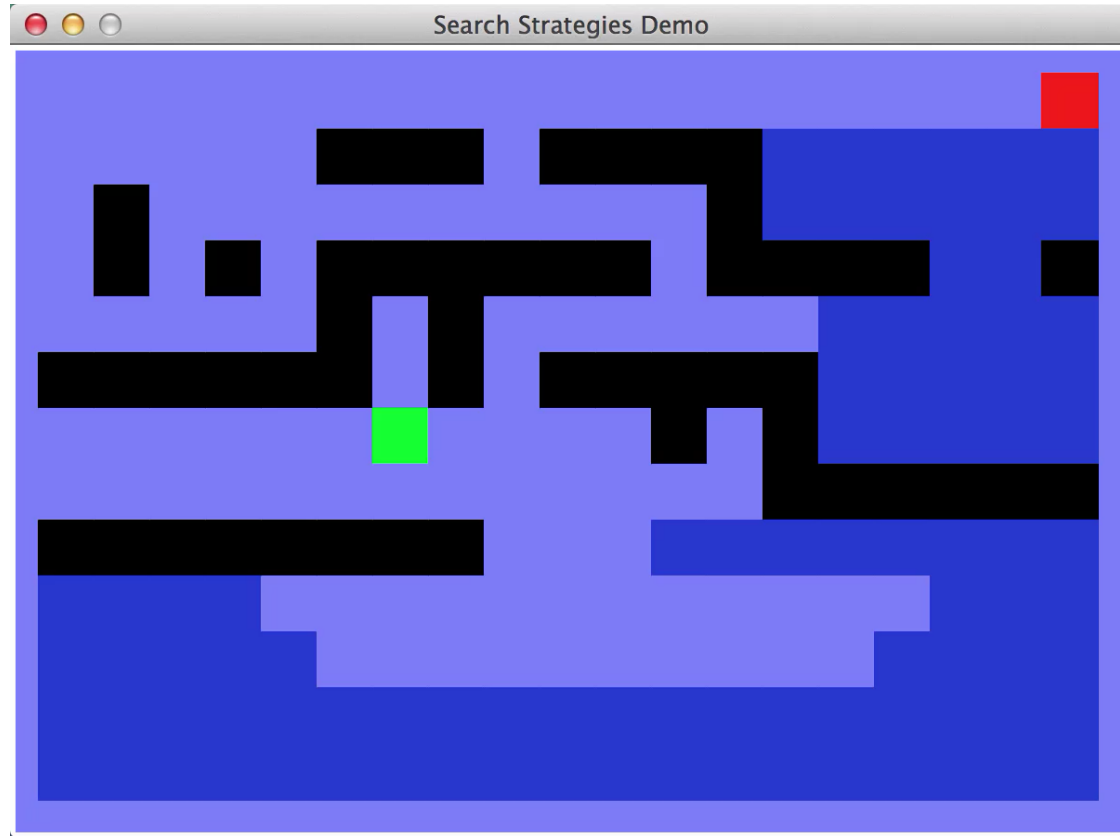
# Video of Demo Empty Greedy

Attendance Code:  
123



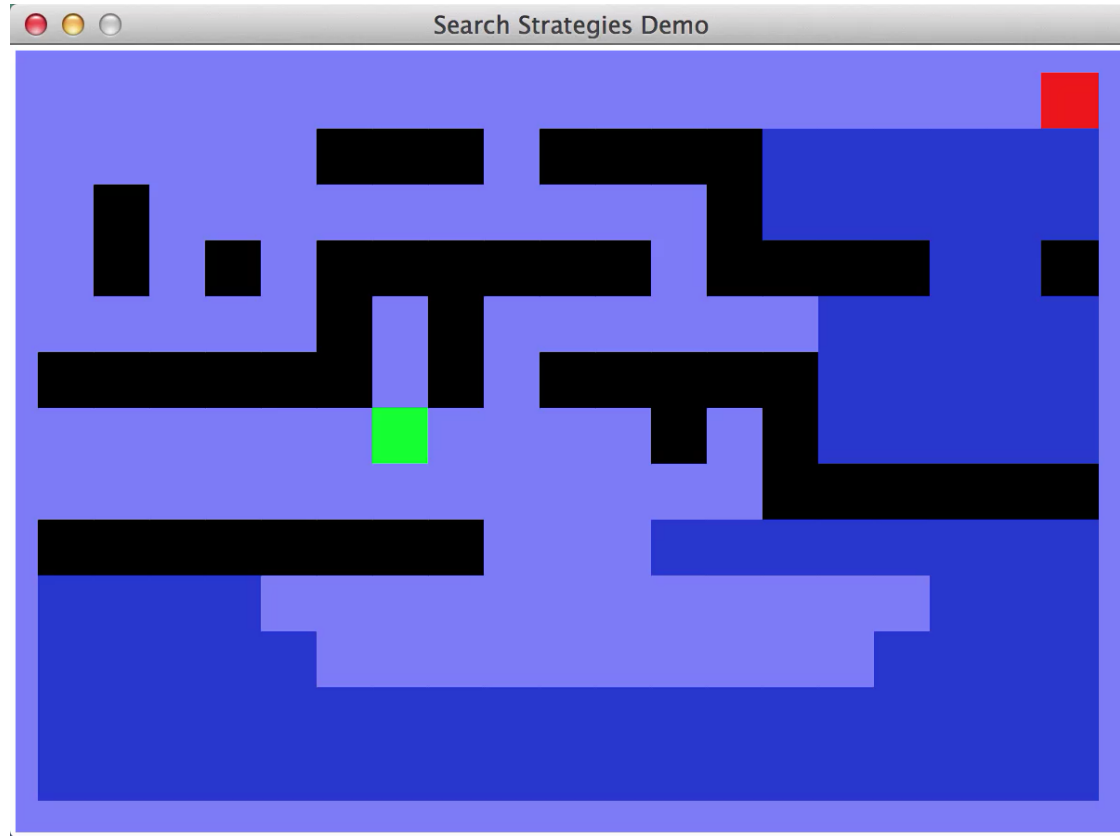
# Video of Demo Maze with Deep/Shallow Water DFS, BFS, or UCS? (part 1)

Attendance Code:  
123



# Video of Demo Maze with Deep/Shallow Water DFS, BFS, or UCS? (part 2)

Attendance Code:  
123

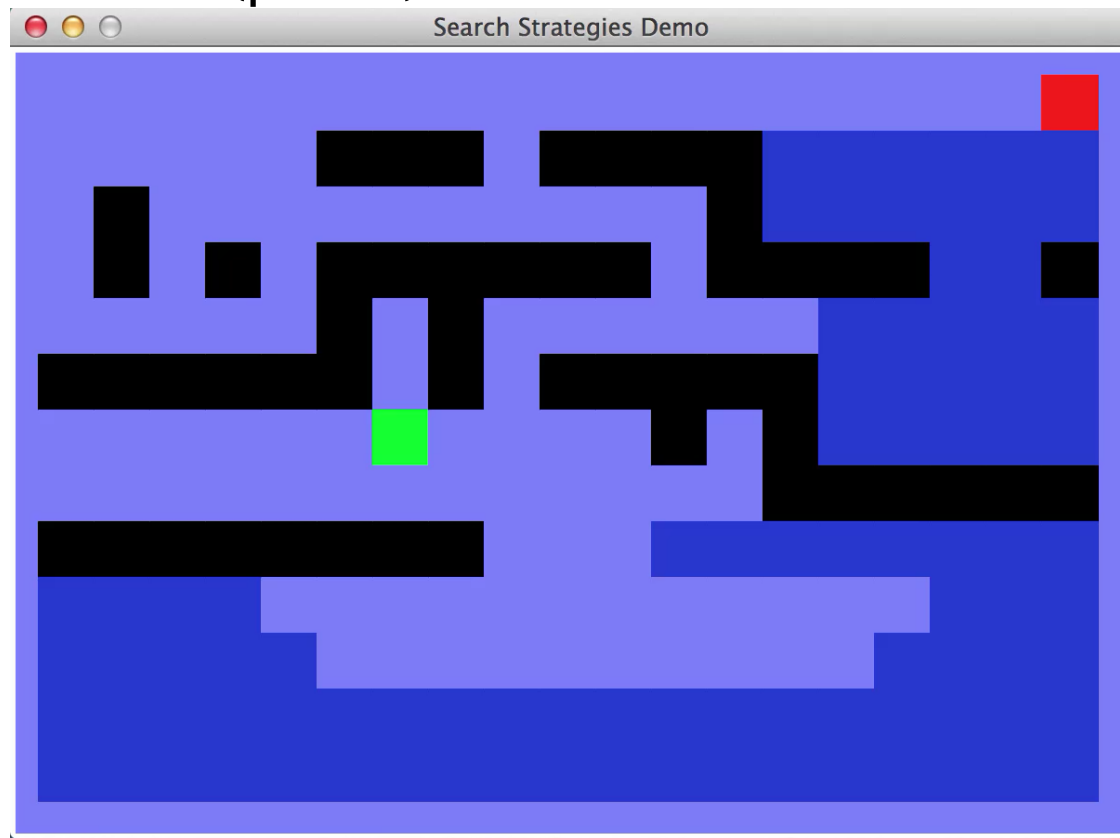




# Video of Demo Maze with Deep/Shallow Water

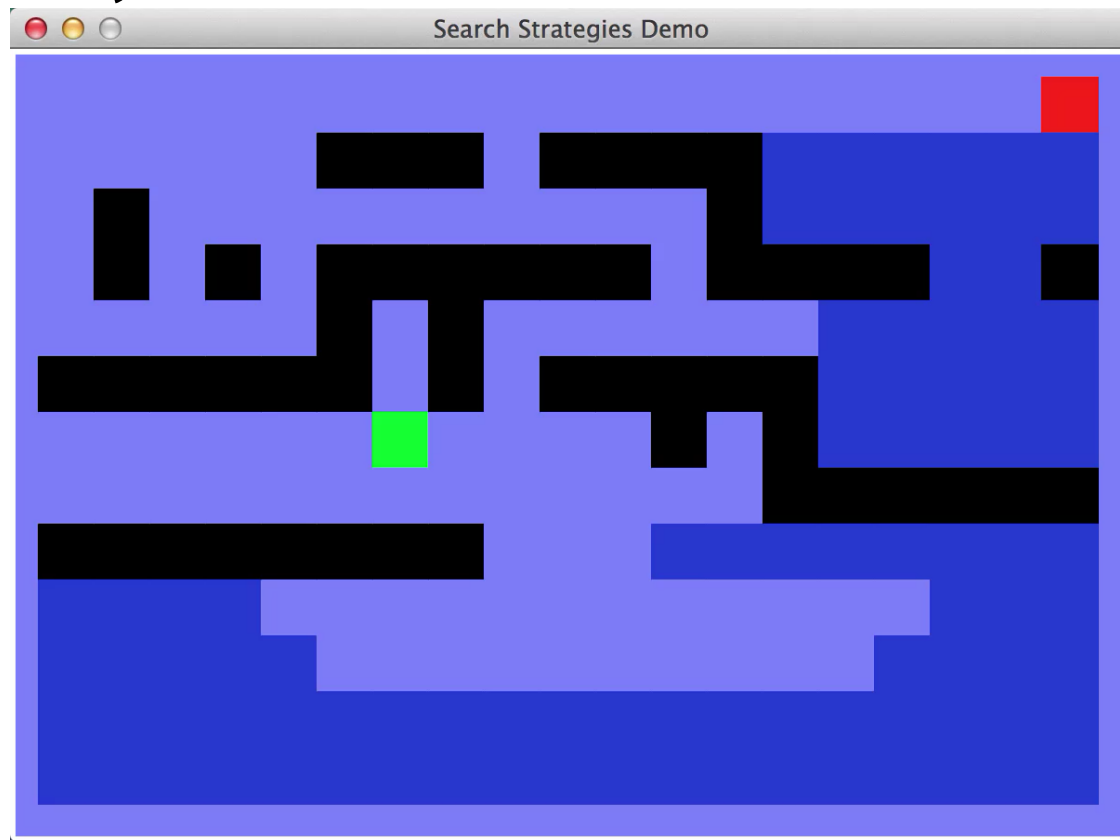
## DFS, BFS, or UCS? (part 3)

Attendance Code:  
123



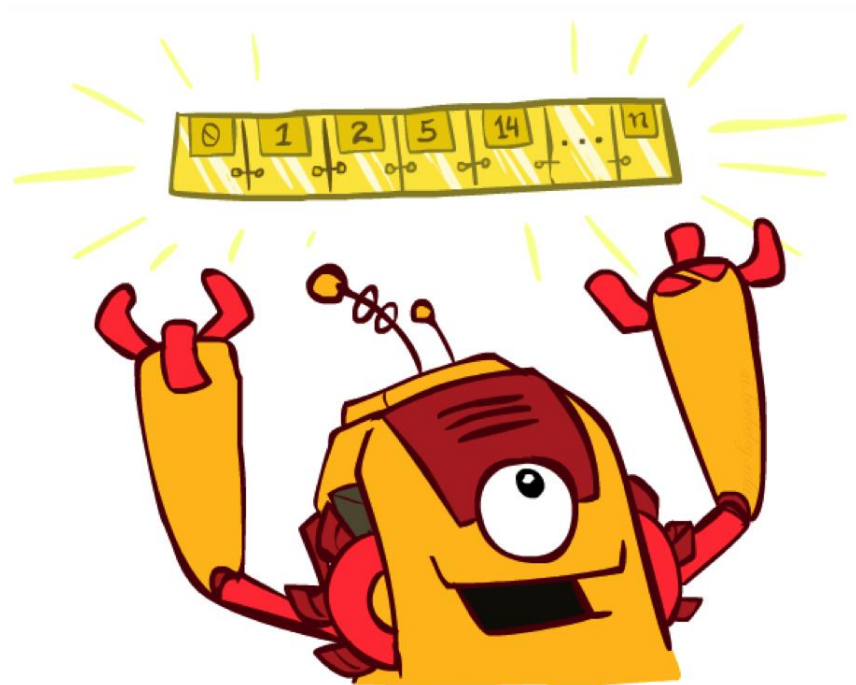
# Video of Demo Maze with Deep/Shallow Water Greedy? (part 4)

Attendance Code:  
123



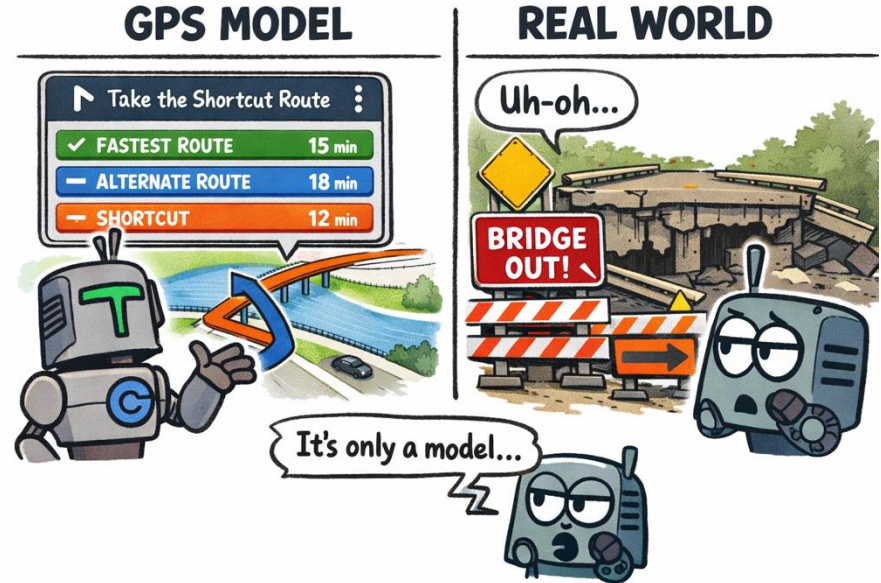
# The One Queue

- All these search algorithms are the same except for fringe strategies
  - Conceptually, all fringes are priority queues (i.e. collections of nodes with attached priorities)
  - Practically, for DFS and BFS, you can avoid the  $\log(n)$  overhead from an actual priority queue, by using stacks and queues
  - Can even code one implementation that takes a variable queuing object



# Search and Models

- Search operates over models of the world
- The agent doesn't actually try all the plans out in the real world!
- Planning is all “in simulation”
- Your search is only as good as your models...



Generated by chatGPT

# Questions

- What's the difference between iterative deepening and DFS?
- Among BFS, DFS, and UCS, which one finds the optimal solution regarding total cost?
- What if minimum arc cost is negative? Does UCS work?

# What's next?

- **Informed** Search!
  - Greedy search
  - A\* algorithm